انجمن علمی مهندسی کامپیوتر
دانشگاه مازندران

به نام خدا

پروژهٔ نهایی

انتقال داده ها

نام و نام خانوادگی اعضاء گروه:

سروش ملکی ۴۰۲۱۳۱۶۰۲۸۱۸۳۷
علی آقابراری کاظمی ۴۰۲۱۳۱۶۰۲۸۱۸۱۰

نام استاد:

دکتر عبدی

۱۰ تیر ۱۴۰۵

فهرست مطالب

۱ مقدمه ۲

۲ اهداف پروژه ۲

۳ معرفی ابزارها ۲
۱.۳ Wireshark ۲
۲.۳ Python ۳
۳.۳ Scapy ۳

٤ معماری و محیط آزمایش ۳

۵ تحلیل ترافیک شبکه ۳
۱.۵ تحلیل پروتکل DNS ۴
۲.۵ تحلیل مکانیزم TCP Three-Way Handshake ۴
۳.۵ تحلیل ترافیک HTTP و HTTPS ۵
۴.۵ تحلیل ترافیک ICMP ۷
۵.۵ تحلیل ترافیک UDP ۷
۶.۵ بررسی Broadcast و Multicast ۸
۷.۵ تحلیل دانلود فایل و جریان ترافیک ۹

٦ پیاده سازی تحلیل ترافیک با Scapy ۹
۱.۶ ضبط بسته های شبکه ۹

۷ نتایج و تحلیل ۱۲
۱.۷ مقایسه عملکرد پروتکل‌های TCP و UDP ۱۲

۸ چالش ها و مشکلات ۱۳

۹ جمع بندی ۱۳

۱۰ منابع ۱۳

۱ مقدمه

در شبکه های کامپیوتری، حجم بسیار زیادی از داده ها در قالب بسته های شبکه میان تجهیزات مختلف جا به جا می شوند.
تحلیل این بسته ها یکی از مهم ترین روش های درک نحوهٔ عملکرد پروتکل های شبکه، شناسایی مشکلات ارتباطی و بررسی رفتار برنامه های کاربردی است. امروزه ابزار هایی مانند Wireshark امکان مشاهده و تحلیل دقیق ترافیک عبوری از شبکه را فراهم می کنند و به مدیران شبکه، کارشناسان امنیت و مهندسان کامپیوتر کمک می کنند تا ساختار ارتباطات شبکه را بهتر درک کنند.

در این پروژه با استفاده از نرم افزار Wireshark و کتابخانهٔ Scapy در زبان برنامه نویسی Python اقدام به ضبط، مشاهده و تحلیل ترافیک شبکه شده است. همچنین رفتار پروتکل های مهمی مانند UDP ، TCP ، DNS و HTTP/HTTPS بررسی شده و نحوهٔ برقراری ارتباط میان میزبان ها مورد تحلیل قرار گرفته است.

۲ اهداف پروژه

اهداف اصلی این پروژه عبارت اند از:

- آشنایی عملی با فرایند ضبط و تحلیل ترافیک شبکه
- یادگیری نحوهٔ استفاده از نرم افزار Wireshark
- بررسی عملکرد پروتکل های مختلف شبکه
- تحلیل فرایند نام گذاری دامنه ها توسط DNS
- مشاهده و تحلیل مکانیزم TCP Three-Way-Handshake
- مقایسهٔ ویژگی های پروتکل TCP و UDP
- شناسایی بسته های Broadcast و Multicast
- آشنایی با کتابخانهٔ Scapy و کاربرد آن در تحلیل ترافیک شبکه
- توسعهٔ مهارت تحلیل و مستند سازی آزمایش های شبکه

۳ معرفی ابزارها

۱.۳ Wireshark

Wireshark یکی از شناخته شده ترین ابزار های تحلیل ترافیک شبکه است که امکان مشاهدهٔ بسته های عبوری از رابط های شبکه را فراهم می کند. این نرم افزار قادر است اطلاعات مربوط به لایه های مختلف شبکه را نمایش داده و جزئیات پروتکل های گوناگون را در اختیار کاربر قرار دهد.
در این پروژه از Wireshark برای ضبط بسته های شبکه و تحلیل پروتکل هایی نظیر DNS ، TCP ، UDP و HTTP/HTTPS استفاده شده است.

Python ۲.۳

Python یکی از پرکاربرد ترین زبان های برنامه نویسی در حوزهٔ شبکه، امنیت و تحلیل داده محسوب می شود. سادگی نگارش، وجود کتابخانه های متعدد و قابلیت توسعهٔ سریع برنامه ها باعث شده است که این زبان در بسیاری از پروژه های شبکه مورد استفاده قرار گیرد.
در این پروژه از Python به همراه کتابخانهٔ Scapy برای تحلیل بسته های شبکه استفاده شده است.

Scapy ۳.۳

Scapy یک کتابخانهٔ قدرتمند برای ایجاد، ارسال، دریافت و تحلیل بسته های شبکه است. این ابزار امکان دسترسی مستقیم به لایه های مختلف پروتکل های شبکه را فراهم کرده و به کاربران اجازه می دهد بسته های سفارشی تولید کنند.
در این پروژه از Scapy برای ضبط بسته های شبکه، شناسایی نوع پروتکل ها و ارسال بسته های آزمایشی استفاده شده است.

٤ معماری و محیط آزمایش

آزمایش های این پروژه بر روی یک رایانهٔ شخصی متصل به شبکهٔ اینترنت انجام شده است. سیستم از طریق رابط Wi-fi/Tetherd Ethernet به روتر محلی متصل بوده و ترافیک شبکه با استفاده از نرم افزار Wireshark ضبط شده است.
مشخصات محیط آزمایش به شرح زیر است:

- سیستم عامل: $Windows\ 10$
- ابزار تحلیل ترافیک: $Wireshark$
- زبان برنامه نویسی: $Python\ 3.13.1$
- کتابخانهٔ تحلیل شبکه: $Scapy$
- نوع اتصال: $Wi-fi/Tetherd\ Ethernet$

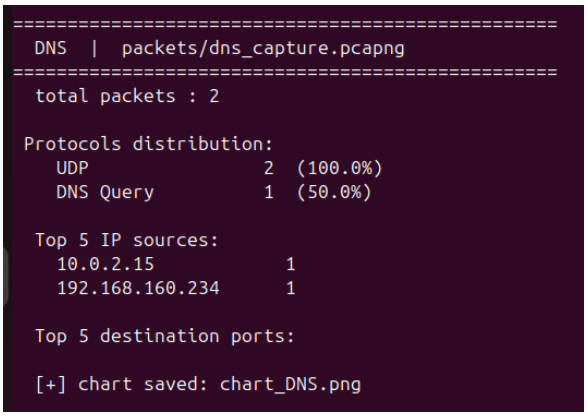
٥ تحلیل ترافیک شبکه

در این بخش، به تحلیل بسته های جمع آوری شده توسط Wireshark در سناریو های مختلف می پردازیم.این تحلیل شامل بررسی پروتکل های پرکاربرد، دست دادن TCP و ترافیک وبسایت ها می باشد.

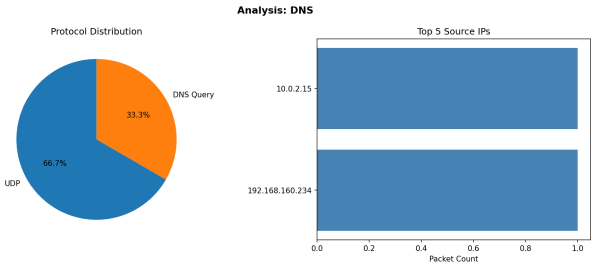
۱.۵ تحلیل پروتکل DNS

پروتکل نام دامنه[1] به عنوان دفترچه تلفن اینترنت عمل می کند. وقتی کاربر آدرس وبسایت (مثلاً google.com) را وارد می کند، کامپیوتر نمی داند که این آدرس کجا می باشد؛ بنابراین، درخواستی به سرور DNS ارسال می کند تا آن نام را به یک آدرس (مثلاً ۷۸.۱۸۵.۲۵۰.۱۴۲) ترجمه کند. با توجه به این تعاریف به نظر شما این پروتکل بر بستر کدام پروتکل لایهٔ انتقال می باشد و چرا؟ این سوالی ایست که در ادامه با بررسی این بخش به آن پاسخ خواهیم داد.

طبق گزارشات، برای بررسی فرآیند نام‌گذاری دامنه، از دستور dig google.com استفاده شد که منجر به ثبت ۲ بسته در نهایت شد. با بررسی بسته‌ها به این نتیجه رسیدیم که ٪۶۶.۷ از ترافیک مربوط به لایه انتقال UDP و ٪۳۳.۳ مربوط به پرس‌وجوی DNS است. این نشان می‌دهد که پرس‌وجوی DNS به‌صورت پیش‌فرض از پروتکل UDP روی پورت ۵۳ استفاده می‌کند چرا که نیاز به یک ارتباط سریع و بدون تأخیر دارد. همچنین آی‌پی مبدأ ۱۰.۰.۲.۱۵ به‌عنوان فرستنده درخواست شناسایی شد (شکل ۱ب).



(آ) خروجی اسکریپت و لاگ ترمینال



(ب) نمودار توزیع پروتکل ها

شکل ۱: تحلیل آماری بسته های DNS

۲.۵ تحلیل مکانیزم TCP Three-Way Handshake

پروتکل کنترل انتقال[2] یک پروتکل **اتصال گرا** و **قابل اعتماد** است. قبل از ارسال هر داده ای، فرستنده و گیرنده باید با یکدیگر فرایندی اصطلاحاً تحت عنوان **دست دادن**[3] را طی کنند تا مطمئن شوند که ارتباط برقرار است. این کار به ترتیب معمولاً شامل ۳ مرحله می باشد:

۱. SYN یا همان ارسال درخواست اولیه

۲. SYN-ACK که پاسخ به درخواست و همچنین درخواست به طرف دیگر

۳. ACK که دریافت نهایی و معمولاً شروع ارتباط را نتیجه می دهد.

این را نیز اشاره کنیم که وقتی از HTTPS استفاده می کنیم، لایه ای به نام TLS بر روی TCP سوار خواهد شد تا داده های ارسالی را رمزنگاری کند.

برای مشاهده فرآیند دست‌دادن TCP، از دستور curl google.com استفاده شد. در این سناریو ۱۵ بسته ثبت شد که ٪۱۰۰ آن‌ها از نوع TCP بودند. تحلیل پورت‌های مقصد نشان داد که بیشترین ترافیک به سمت پورت ۸۰ (HTTP) و پورت ۴۴۳ (HTTPS) بوده

[1] Domain Name Server
[2] Transport Control Protocol
[3] Handshake

است. همچنین آی‌پی مبدأ ‘۱۰.۰.۲.۱۵’ با ارسال ۸ بسته و آی‌پی مقصد ‘۲۱۶.۲۳۹.۳۸.۱۲۰’ با دریافت ۵ بسته، نشان‌دهنده برقراری ارتباط کامل بین کلاینت و سرور گوگل است. در این فرآیند، توالی بسته‌های SYN، SYN-ACK و ACK به‌خوبی مشاهده شد. نتایج این تحلیل در شکل ۲ب قابل مشاهده است.

```
================================================
  TCP/TLS  |  packets/tcp_capture.pcapng
================================================
  total packets : 15

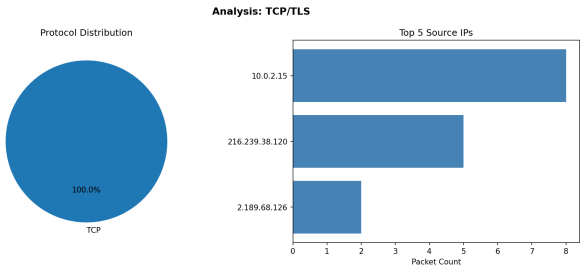
 Protocols distribution:
   TCP              15  (100.0%)

  Top 5 IP sources:
    10.0.2.15           8
    216.239.38.120      5
    2.189.68.126        2

  Top 5 destination ports:
    PORT 80       6
    PORT 58824    5
    PORT 443      2
    PORT 35612    2

  [+] chart saved: chart_TCP_TLS.png
```

(آ) لاگ ترمینال بخش مربوط به TCP که شامل ۱۵ بسته می باشد.



(ب) نمودار توزیع آی پی های مبدأ

شکل ۲: تحلیل مکانیزم TCP و ترافیک HTTPS

۳.۵ تحلیل ترافیک HTTP و HTTPS

پروتکل انتقال ابرمتن[۴] یک پروتکل بی حالت[۵] در لایهٔ کاربرد[۶] می باشد که برای انتقال صفحات وب، تصاویر و فایل ها استفاده می شود. در این پروتکل، مرورگر یک درخواست[۷] می فرستد و سرور یک پاسخ[۸] بر می گرداند. اما نکتهٔ جالب این است که داده ها به صورت **متن ساده**[۹] جابجا می شوند اما رمزنگاری نمی شوند.

انتقال داده در پروتکل HTTP درست مانند **فرستادن یک کارت پستال باز** می باشد؛ چرا که هر کسی در مسیر ارتباط اعم از اپراتور مخابراتی یا هکر می تواند محتوای آن را به راحتی بخواند. به همین دلیل، وبسایت های مدرن به سمت پروتکل امن انتقال ابرمتن[۱۰] مهاجرت کرده اند به دلیل اینکه در این پروتکل حالت امن[۱۱] و رمزنگاری شده وجود دارد.

[4] HTTP - HyperText Transfer Protocol
[5] Stateless
[6] Application
[7] Request
[8] Response
[9] Plain Text
[10] HTTPS - HyperText Transfer Protocol Secure
[11] Secure

```
================================================
  HTTP  |  packets/http_capture.pcapng
================================================
  total packets : 839

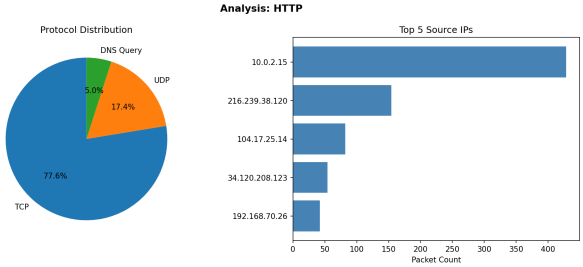
 Protocols distribution:
    TCP              685  (81.6%)
    UDP              154  (18.4%)
    DNS Query         44  (5.2%)

  Top 5 IP sources:
    10.0.2.15            428
    216.239.38.120       154
    104.17.25.14         82
    34.120.208.123       54
    192.168.70.26        42

  Top 5 destination ports:
    PORT 443       271
    PORT 52980     63
    PORT 34230     54
    PORT 80        45
    PORT 44874     43

  [+] chart saved: chart_HTTP.png
```

(آ) لاگ ترمینال شامل ۸۳۹ بسته



(ب) نمودار توزیع پروتکل‌ها و آی‌پی‌های مبدأ در ترافیک HTTP

شکل ۳: تحلیل جریان ترافیک HTTP

برای بررسی ترافیک HTTP، وبسایت http://httpforever.com در مرورگر باز شد. این سناریو دارای بیشترین حجم داده در میان آزمایش‌ها بود و ۸۳۹ بسته ضبط گردید. نمودار توزیع پروتکل‌ها نشان می‌دهد که ٪۸۱.۶ از بسته‌ها از نوع TCP و ٪۱۸.۴ از نوع UDP هستند. حضور پروتکل UDP به‌دلیل پرس‌وجوهای ابتدایی DNS برای پیدا کردن آی‌پی وبسایت است. از نظر آی‌پی‌های مبدأ، آی‌پی ۱۰.۰.۲.۱۵ با ارسال ۴۲۸ بسته، بیشترین نقش را داشته و ارتباطات گسترده‌ای با آی‌پی‌های مقصد ۲۱۶.۲۳۹.۳۸.۱۲۰ (۱۵۴ بسته) و ۱۰۴.۱۷.۲۵.۱۴ (۸۲ بسته) برقرار کرده است. این الگو نشان‌دهنده بارگذاری محتوای وبسایت از سرورهای مختلف است (شکل ۳ب).

۴.۵ تحلیل ترافیک ICMP

پروتکل پیام های کنترل اینترنت[۱۲] در لایهٔ شبکه (لایهٔ ۳) کار می کند و وظیفهٔ اصلی آن، **عیب یابی و گزارش خطا** است. این پروتکل پورت ندارد و مستقیماً روی IP سوار می شود. معروف ترین دستورات کاربردی آن ping که در این پروژه استفاده شد و دستور traceroute می باشد.

```
================================================
  ICMP  |  packets/icmp_capture.pcapng
================================================
  total packets : 8

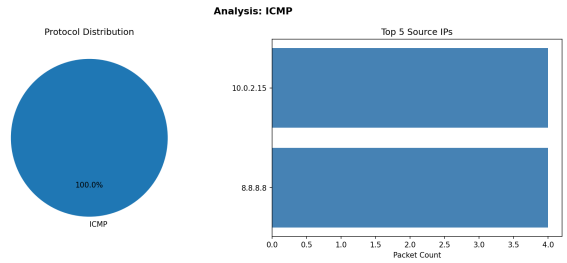
 Protocols distribution:
    ICMP              8  (100.0%)

  Top 5 IP sources:
    10.0.2.15           4
    8.8.8.8             4

  Top 5 destination ports:

  [+] chart saved: chart_ICMP.png
```

(آ) لاگ ترمینال و خلاصهٔ نتیجه از Wireshark که در اینجا شامل ۴ جفت بستهٔ درخواست و پاسخ بود.



(ب) نمودار Pie مربوط به توزیع پروتکل ها و IP های ICMP

شکل ۴: تحلیل بسته‌های ICMP

برای این آزمایش، در ابتدا دستور ping google.com -c 4 اجرا شد و مطابق تصویر ۴آ بسته های Echo Request و Echo Reply که به ترتیب درخواست و پاسخ هستند را نشان می دهد که دقیقاً ۴ جفت می باشند.
اسکریپتی که ما در این پروژه نوشتیم این ۸ بسته را به درستی به عنوان ‘۱۰۰٪ ICMP’ شناسایی و طبقه بندی کرده است.

۵.۵ تحلیل ترافیک UDP

در آزمایشات ما، ترافیک UDP عمدتاً در قالب پرس‌وجوهای DNS مشاهده شد. برخلاف TCP، این پروتکل ارتباط را قبل از ارسال داده برقرار نمی‌کند (بدون دست‌دادن). این ویژگی باعث کاهش تأخیر در پرس‌وجوهای نام‌گذاری می‌شود که برای عملکرد سریع وبگردی حیاتی است.

[12] Internet Control Message Protocol

۶.۵ بررسی Broadcast و Multicast

Broadcast و Multicast دو روش کلیدی برای انتقال داده از یک مبدأ به چندین مقصد در شبکه هستند. درک تفاوت این دو برای تحلیل ترافیک شبکه بسیار حیاتی است و در این بخش به بررسی و شبیه سازی آن ها می پردازیم.

```
==================================================
  ARP   |   packets/arp_capture.pcapng
==================================================
  total packets : 1

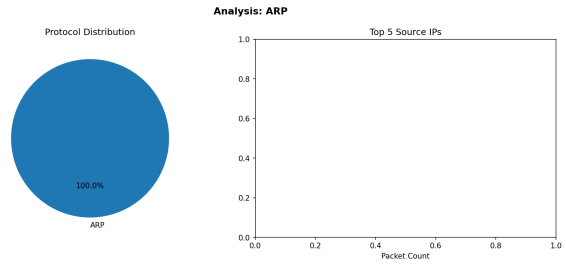
 Protocols distribution:
    ARP                 1   (100.0%)

  Top 5 IP sources:

  Top 5 destination ports:

  [+] chart saved: chart_ARP.png
```

(آ) لاگ ترمینال مربوط به بخش ARP که شامل تنها یک بستهٔ Broadcast بود.



(ب) نمودار و دیدن خالی بودن صحیح میله های IP

شکل ۵: تحلیل ترافیک Broadcast ARP توسط اسکریپت Python

پخش همگانی (Broadcast) به عملی گفته می‌شود که در آن یک بسته، به تمام میزبان‌های موجود در یک زیرشبکه (Subnet) ارسال می‌شود. آدرس IP Broadcast معمولاً به صورت 255.255.255.255 یا آدرس مخصوص زیرشبکه است و در لایه ۲ (لایه پیوند داده)، آدرس MAC مقصد برابر با ff:ff:ff:ff:ff:ff خواهد بود. بارزترین مثالی که در این پروژه با آن روبرو شدیم، پروتکل ARP است.
می توان مکانیزم ARP برای حالت Broadcast درست مانند **فریاد زدن در یک محله** درنظر گرفت؛ در سناریویی که شخصی به دنبال آدرس خانه ای می گردد و برای یافتن آن نام یا آدرس آن خانه را با صدای بلند فریاد می زند. (ARP Request)
در ادامه، همهٔ ساکنان محله (شبکهٔ محلی) این صدا را می شنوند اما فقط صاحب خانه (همان میزبان موردنظر) به او پاسخ می دهد و بقیهٔ ساکنان، بسته را نادیده می گیرند.

در عین حال ارسال Multicast مانند **برگزاری یک جلسهٔ آنلاین** مانند وبینار یا کنفرانس ویدئویی می باشد که شما یک لینک جلسه یا گروه ایجاد می کنید و فقط آن دسته از افرادی که لینک را دریافت کرده اند و به جلسه وارد یا عضو گروه شده اند، می توانند صدای شما را بشنوند و ویدئوی شما را ببینند. این برخلاف Broadcast است چرا که در حالت پخش همگانی شما با یک بلندگو در خیابان فریاد می زنید تا همه بشنوند **حتی کسانی که نمی خواهند بشنوند**. به زبان ساده تر، در پخش گروهی بسته ها تنها به **گروه خاصی از میزبان ها** که به آن گروه مشترک شده اند ارسال می شود.

برای شبیه سازی این رویداد، اسکریپت bm.py که محتویات آن به صورت زیر می باشد، یک بستهٔ ARP با آدرس مقصد ff:ff:ff:ff:ff:ff تولید و درنهایت ارسال کرد.

```python
from scapy.all import Ether, ARP, sendp

pkt = Ether(dst="ff:ff:ff:ff:ff:ff") / ARP(pdst="10.0.2.15")
sendp(pkt, verbose=1)
```

۷.۵ تحلیل دانلود فایل و جریان ترافیک

در شبکه، جریان ترافیک[۱۳] به دنباله ای از بسته ها گفته می شود که از یک مبدأ به یک مقصد خاص و برای یک عملیات مشخص ارسال می شوند. تحلیل این جریان به ما نشان می دهد که برای دانلود یک فایل یا بارگذاری یک وبسایت، ارتباطات چگونه بین سرور ها تقسیم می شوند.

اگرچه یک سناریوی دانلود فایل حجیم به صورت مجزا در این پروژه ضبط نشده است، اما سناریوی مرور وب بهترین الگو برای تحلیل یک جریان پرحجم را نشان می دهد.
با تحلیل نمودار IP های مبدأ در تصویر ۳ نکات زیر قابل دریافت هستند:

- آی‌پی مبدأ '۱۰.۰.۲.۱۵' با ارسال ۴۲۸ بسته، نقش اصلی را در ارسال درخواست‌های متوالی برای دریافت منابع (تصاویر، فایل‌های جانبی) ایفا کرده است.
- آی‌پی‌های مقصد (سرورهای وب) شامل '۲۱۶.۲۳۹.۳۸.۱۲۰' و '۱۰۴.۱۷.۲۵.۱۴' هستند. این الگو نشان‌دهنده تقسیم بار[۱۴] توسط سرویس‌های شبکهٔ تحویل محتوا[۱۵] است که برای تسریع در دریافت فایل‌ها، درخواست کاربر را به بهترین و نزدیک‌ترین سرور هدایت می‌کنند.

۶ پیاده سازی تحلیل ترافیک با Scapy

در این بخش، به بررسی اسکریپت‌های پایتون توسعه‌داده‌شده برای تحلیل بسته‌ها می‌پردازیم.

۱.۶ ضبط بسته های شبکه

اسکریپت final_code.py با استفاده از کتابخانه Scapy فایل‌های pcapng. را خوانده و برای هر بسته، لایه‌های مختلف (از جمله TCP، UDP، ICMP، ARP و DNS) را شناسایی می‌کند. همچنین با استفاده از کتابخانه matplotlib، نمودارهای دایره‌ای و میله‌ای برای نمایش توزیع پروتکل‌ها و آی‌پی‌های مبدأ تولید می‌کند.

```python
from scapy.all import rdpcap, IP, TCP, UDP, DNS, ICMP, ARP
from collections import Counter
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.family'] = 'DejaVu Sans'

#    the files which have been captured
FILES = {
    "HTTP":          "packets/http_capture.pcapng",
    "ARP":           "packets/arp_capture.pcapng",
    "ICMP":          "packets/icmp_capture.pcapng",
    "DNS":           "packets/dns_capture.pcapng",
    "TCP/TLS":       "packets/tcp_capture.pcapng",
}

#    Analysis of the pcap file
def analyze(label, path):
    print(f"\n{'='*50}")
    print(f"  {label}  |  {path}")
    print('='*50)

    try:
```

[13] Traffic Flow
[14] Load Balancing
[15] CDN - Content Delivery Network

```
        packets = rdpcap(path)
    except Exception as e:
        print(f"  [!] Error in reading file: {e}")
        return None

    total     = len(packets)
    protocols = Counter()
    src_ips   = Counter()
    dst_ports = Counter()

    for p in packets:
        # Transport Layer Protocol
        if p.haslayer(TCP):
            protocols['TCP'] += 1
            dst_ports[p[TCP].dport] += 1
        elif p.haslayer(UDP):
            protocols['UDP'] += 1
        elif p.haslayer(ICMP):
            protocols['ICMP'] += 1
        elif p.haslayer(ARP):
            protocols['ARP'] += 1

        # DNS (On UDP/TCP both may be possible)
        if p.haslayer(DNS) and p[DNS].qr == 0:
            protocols['DNS Query'] += 1

        # IP of the Origin
        if p.haslayer(IP):
            src_ips[p[IP].src] += 1

    #    Printing the statistics
    print(f"  # of all packets: {total}")

    print("\n  Protocols' Distribution:")
    for proto, count in protocols.most_common():
        pct = round(count / total * 100, 1) if total else 0
        print(f"    {proto:<14} {count:>5}  ({pct}%)")

    print("\n  Top 5 Origin IPs:")
    for ip, count in src_ips.most_common(5):
        print(f"    {ip:<20} {count}")

    print("\n  Top 5 Destination Port:")
    for port, count in dst_ports.most_common(5):
        print(f"    PORT {port:<8} {count}")

    return {"label": label, "protocols": protocols,
            "src_ips": src_ips, "total": total}


#    Tracing a chart for a result
def plot(result):
```

```
    if result is None or result["total"] == 0:
        return

    label     = result["label"]
    protocols = result["protocols"]
    src_ips   = result["src_ips"]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
    fig.suptitle(f"Analysis: {label}", fontsize=13, fontweight='bold')

    # Protocols' Pie chart
    if protocols:
        ax1.pie(
            protocols.values(),
            labels=protocols.keys(),
            autopct='%1.1f%%',
            startangle=90
        )
    ax1.set_title("Protocol Distribution")

    # Bar chart of the Top 5 Origin IPs
    top_ips  = [x[0] for x in src_ips.most_common(5)]
    top_cnts = [x[1] for x in src_ips.most_common(5)]
    if top_ips:
        ax2.barh(top_ips, top_cnts, color="steelblue")
        ax2.invert_yaxis()
    ax2.set_xlabel("Packet Count")
    ax2.set_title("Top 5 Source IPs")

    plt.tight_layout()
    filename = f"chart_{label.replace('/', '_')}.png"
    plt.savefig(filename, dpi=150, bbox_inches='tight')
    plt.close()
    print(f"\n  [+] The Chart is now saved and named: {filename}")


#    Main Execution
if __name__ == "__main__":
    for label, path in FILES.items():
        result = analyze(label, path)
        plot(result)

    print("\n\n all files are now processed successfully (^_^)")
```

۷ نتایج و تحلیل

نتایج حاصل از بررسی فایل‌های pcapng. در جدول ۱ خلاصه شده است.

جدول ۱: خلاصه آماری ترافیک شبکه در سناریوهای مختلف

نوع ترافیک	فایل pcapng.	تعداد بسته‌ها	پروتکل غالب
DNS	dns_capture.pcapng	۲	UDP
ICMP	icmp_capture.pcapng	۸	ICMP
TCP/TLS	tcp_capture.pcapng	۱۵	TCP
HTTP	http_capture.pcapng	۸۳۹	TCP
ARP	arp_capture.pcapng	۱	ARP

۱.۷ مقایسه عملکرد پروتکل‌های TCP و UDP

مهم‌ترین بخش تحلیل در اینجا، مقایسه عملکرد دو پروتکل حمل‌ونقل است که در جدول ۲ آمده است:

جدول ۲: مقایسه پروتکل‌های TCP و UDP

ویژگی	TCP	UDP
نوع ارتباط	اتصال‌گرا (با دست‌دادن)	بدون اتصال
قابلیت اطمینان	بالا (تضمین تحویل داده)	پایین (فقط ارسال می‌کند)
سرعت	کندتر (به‌دلیل کنترل جریان و خطا)	سریعتر
مورد استفاده	وبگردی (HTTP)، ایمیل (SMTP)	پرس‌وجوی DNS ، استریم ویدیو

همانطور که در نتایج آزمایش HTTP مشاهده شد، حجم بالای بسته‌های TCP نشان می‌دهد که برای انتقال محتوای وب، نیاز به ارتباط قابل اعتماد، کنترل ازدحام و بازبینی خطا وجود دارد. در مقابل، UDP به‌دلیل سبک‌بودن و سرعت بالا، بهترین گزینه برای DNS است.

۸ چالش ها و مشکلات

در طول انجام پروژه، چالش‌های زیر وجود داشت:

- تنظیم فیلترهای Wireshark: برای جدا کردن ترافیک خاص (مثلاً فقط DNS)، نیاز به استفاده صحیح از فیلترهای نمایش Wireshark (نظیر dns، tcp و ...) بود که در ابتدا نیازمند یادگیری بود.
- تحلیل بسته‌های ARP: به‌دلیل عدم وجود لایه IP در بسته‌های ARP، اسکریپت final_code.py نمی‌توانست آی‌پی مبدأ را شناسایی کند که این موضوع در ابتدا باعث سردرگمی شد. اما پس از بررسی ساختار لایه‌ها، مشخص شد که این رفتار عادی ARP است.
- نصب و راه‌اندازی Scapy: وابستگی‌های کتابخانه Scapy برای ضبط بسته‌ها در Windows نیازمند تنظیمات اولیه (نصب Npcap با تنظیمات خاص) بود که در نهایت نیز منجر به استفاده از سیستم عامل لینوکس توزیع اوبونتو برای انجام پروژه شد.

۹ جمع بندی

در این پروژه، با ادغام دانش تئوری شبکه و ابزارهای عملی، چرخه کامل تحلیل ترافیک شبکه شامل "ضبط، مشاهده، پردازش و تحلیل" پیاده‌سازی شد. مشاهدات به‌خوبی نشان داد:

- پروتکل TCP با مکانیزم اتصال‌گرا (دست‌دادن سه‌مرحله‌ای) و تضمین تحویل، ستون فقرات وب و HTTPS است.
- پروتکل UDP با معماری بدون اتصال، سرعت مورد نیاز DNS را فراهم می‌کند.
- پروتکل ARP در لایه دوم شبکه با ارسال پخش‌همگانی، زیرساخت ارتباطات محلی را مهیا می‌سازد.

کتابخانه Scapy در Python به‌وضوح نشان داد که چقدر قدرتمند است و می‌تواند نه‌تنها برای تحلیل، بلکه برای تولید ترافیک سفارشی (مانند ARP Broadcast) مورد استفاده قرار گیرد. در ادامه مسیر، می‌توان این پروژه را با بررسی پروتکل‌های مسیریابی مثل OSPF و یا تحلیل جریان‌های TCP/UDP در محیط‌های چندمیزبانه توسعه داد.

۱۰ منابع

- وب‌سایت رسمی مستندات Wireshark: https://www.wireshark.org/docs/
- مستندات کتابخانه Scapy: https://scapy.readthedocs.io/
- کتاب مرجع: Computer Networking: A Top-Down Approach, Kurose & Ross

پایان